

Blocco 2 - Chiavi, domini e integrità

Trasformare il modello in regole verificabili

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Rappresentazione iniziale

Partiamo da una soluzione semplice e concreta, prima di introdurre il modello generale.

Perché questo blocco

Problema didattico

Un modello senza vincoli consente dati tecnicamente inseribili ma concettualmente impossibili: ordini senza cliente, sconti del 130%, quantità negative, due clienti con la stessa email o spedizioni consegnate prima di essere partite.

Idea guida

Prima osserviamo una rappresentazione naturale, poi ne discutiamo i limiti e solo dopo formalizziamo.

Tabella compilabile senza regole

Situazione concreta

Dopo aver separato i dati in tabelle, potremmo limitarci a scrivere colonne e righe. Una tabella ordini senza regole accetta quasi qualunque valore.

order_id	customer_id	status	quantity	discount
1	10	completed	2	5
2	999	closed	-3	130
3	10	pending	0	0

Limiti della rappresentazione iniziale

- ▶ cliente inesistente;
- ▶ stato non ammesso;
- ▶ quantità impossibile;
- ▶ sconto fuori dominio;
- ▶ nessuna protezione contro duplicati.

2. Soluzione più generale

Riorganizziamo l'informazione in una forma più flessibile e controllabile.

Principio

La soluzione generale è far diventare esplicite le regole essenziali: identità delle righe, valori ammessi, riferimenti validi e obbligatorietà.

Esempio di riorganizzazione

- ▶ PRIMARY KEY: identifica una riga.
- ▶ FOREIGN KEY: collega una riga a una riga esistente.
- ▶ CHECK: limita i valori ammessi.
- ▶ NOT NULL: rende obbligatoria una informazione.
- ▶ UNIQUE: impedisce duplicati business.

Processo di trasformazione

- ➊ trovare l'identificatore di ogni relazione;
- ➋ stabilire domini e valori ammessi;
- ➌ rendere espliciti i riferimenti tra tabelle;
- ➍ decidere dove NULL è accettabile;
- ➎ testare righe valide e non valide.

Analogia o caso esplicativo

Un edificio non si limita a registrare chi entra: usa badge, tornelli e aree autorizzate. I vincoli fanno lo stesso nei dati: non correggono il processo aziendale, ma impediscono che entrino stati che il processo considera impossibili.

3. Formalizzazione

Diamo un nome preciso ai concetti emersi dall'esempio.

- ▶ integrità di entità: ogni riga è identificabile;
- ▶ integrità referenziale: i riferimenti puntano a righe esistenti;
- ▶ integrità di dominio: i valori appartengono all'insieme ammesso;
- ▶ vincoli business: regole specifiche del dominio.

- ▶ **Chiave candidata:** attributo o insieme di attributi che identifica una riga.
- ▶ **Chiave primaria:** identificatore principale scelto per i riferimenti.
- ▶ **Chiave esterna:** collegamento verso una riga esistente.
- ▶ **Dominio:** valori ammessi per un attributo.
- ▶ **Vincolo:** regola che il database deve far rispettare.

Come riconoscere una buona soluzione

- ▶ ogni relazione ha una chiave primaria non ambigua;
- ▶ le chiavi esterne riflettono relazioni obbligatorie o opzionali;
- ▶ i domini impediscono valori fuori significato;
- ▶ le decisioni su cancellazione e storia sono documentate.

4. Esempio guidato

Applichiamo il modello generale a un caso piccolo, leggendo ogni passaggio.

Esempio: situazione iniziale

Domanda o frase di dominio

Caso: una riga ordine deve riferirsi a un ordine esistente e a un prodotto esistente; la quantità deve essere positiva; lo sconto deve essere tra 0 e 100.

Esempio: ragionamento

- ▶ l'identità della riga può essere (`order_id`, `product_id`) oppure una chiave surrogate;
- ▶ la riga non ha senso senza ordine e prodotto;
- ▶ il prezzo in riga ordine va storicizzato, non letto sempre dal prodotto corrente;
- ▶ la quantità zero non rappresenta una vendita.

Esempio: vincoli attesi

Regola	Vincolo
cliente identificabile	PRIMARY KEY(customer_id)
email non duplicata	UNIQUE(email)
ordine con cliente valido	FOREIGN KEY(customer_id)
quantità positiva	CHECK(quantity > 0)
sconto valido	CHECK(discount_pct >= 0 AND discount_pct < 100)

Errori frequenti da prevenire

- ▶ lasciare tutto nullable per evitare decisioni;
- ▶ usare un campo descrittivo come chiave primaria;
- ▶ affidare tutte le regole al codice applicativo;
- ▶ introdurre una chiave surrogate e dimenticare l'unicità business.

5. Pratica guidata

Ripetiamo lo stesso processo su un problema diverso: rappresentazione iniziale, limiti, riorganizzazione, soluzione.

Contesto

Riprendendo il modello del blocco 1, il responsabile dati chiede che il database rifiuti errori comuni prima che arrivino nei report.

Esercitazione: specifica corretta

- ▶ input: relazioni candidate del blocco 1;
- ▶ output: chiavi primarie, chiavi candidate, chiavi esterne, domini e vincoli;
- ▶ vincolo: distinguere regole implementabili subito e regole da gestire a livello applicativo;
- ▶ vincolo: ogni NULL deve avere una motivazione di dominio.

Esercitazione: definizione della soluzione

- ① usare surrogate key per riferimenti stabili quando la chiave naturale può cambiare;
- ② conservare UNIQUE su email, SKU o codici esterni quando il business lo richiede;
- ③ rendere obbligatorie le relazioni senza cui il fatto non esiste;
- ④ decidere politiche conservative per cancellazioni di fatti storici.

Implementazione completa: dizionario dei vincoli

- ▶ customers: PK customer_id, UNIQUE email, country NOT NULL
- ▶ products: PK product_id, UNIQUE sku, unit_price ≥ 0
- ▶ orders: PK order_id, FK obbligatoria verso customers
- ▶ order_items: FK verso orders e products, quantity > 0
- ▶ shipments: FK verso orders, delivered_at opzionale fino alla consegna

Esercitazione: organizzazione dei 90 minuti

- ▶ 20 min chiavi e identificatori business;
- ▶ 25 min domini e obbligatorietà;
- ▶ 25 min chiavi esterne e casi limite;
- ▶ 20 min discussione sulle regole non implementabili con vincoli semplici.

- ▶ confrontare almeno due soluzioni quando esistono alternative;
- ▶ chiedere quali assunzioni cambierebbero la soluzione;
- ▶ separare errori di sintassi, errori di modello ed errori di interpretazione.

- ▶ un vincolo non è burocrazia: è una frase del dominio resa controllabile;
- ▶ una chiave surrogate semplifica i join, ma non protegge da duplicati business;
- ▶ NULL va usato per informazione assente, non per indecisione progettuale.